

CS4780 - Lab 3: Plotting and Shaping

Contents

Getting started	1
Loading the data	1
Plotting	2
Adding colour and legend	3
Exercise 1	4
Exercise 2	5
Data wrangling	5
A more elegant way to merge data	5
Multiplot	7
Wide vs long (or narrow) format	7
Reshaping the data	7
The ‘ggplot’ library	8
Exercise 3	8

Submitting your Lab assignment

Add the answers for the individual exercises to `Lab03_Worksheet.txt` available on BlackBoard. You only need to add the code to the worksheet that generates the solution. You do **not** need to submit any plots or tables.

At the end of the lab, please submit the filled worksheet as **plain text** via the corresponding link on Blackboard. *No other formats will be considered.*

After submission deadline, the answers to the exercises will be published as `Lab03_Solutions.txt` on Blackboard.

In this lab, you will learn how to do basic data handling and plotting in R. The data that we will be using is the number of Google scholar publications for 5 different search terms.

Getting started

Loading the data

First, we load the various data files from CSV. You can download `Lab03_Data.zip` from BlackBoard . Unzip the archive and make sure you are in the correct directory or use the full path to the file. To change the working directory use `setwd([...])`. To find your current directory use `getwd()`.

```
artificial_social_agent <-  
  read.csv(file="data/artificial_social_agent.csv", header=TRUE, sep=",")  
embodied_conversational_agent <-
```

```
read.csv(file="data/embodied_conversational_agent.csv", header=TRUE, sep=",")
social_robot <- read.csv(file="data/social_robot.csv", header=TRUE, sep=",")
virtual_character <- read.csv(file="data/virtual_character.csv", header=TRUE, sep=",")
virtual_human <- read.csv(file="data/virtual_human.csv", header=TRUE, sep=",")
```

You can now look at your data set by double-clicking the name in the variable browser in R Studio:

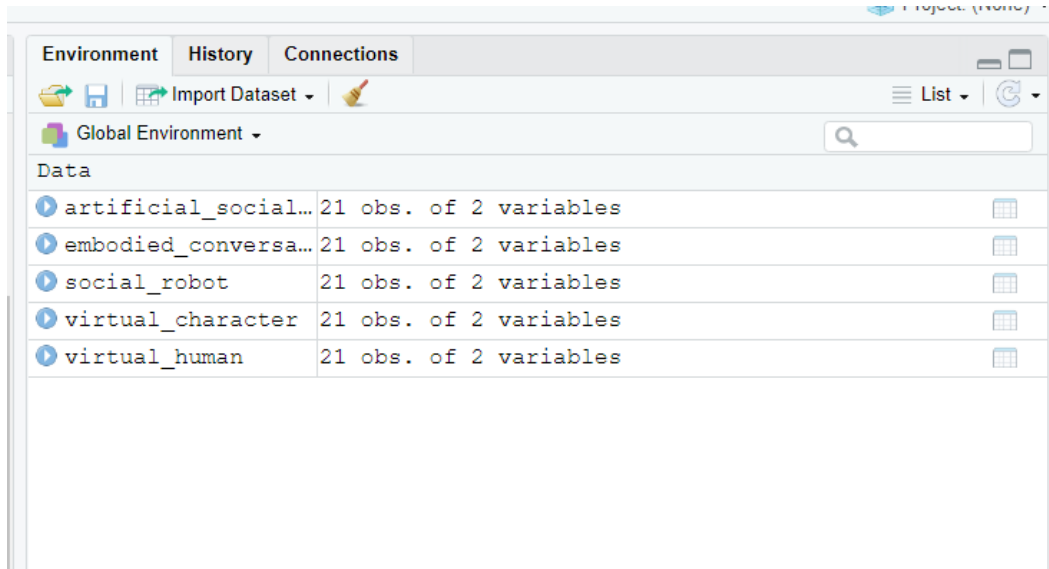


Figure 1: R Studio Environment Browser

A simple way to check your data is to let R calculate some summary statistics using `summary`:

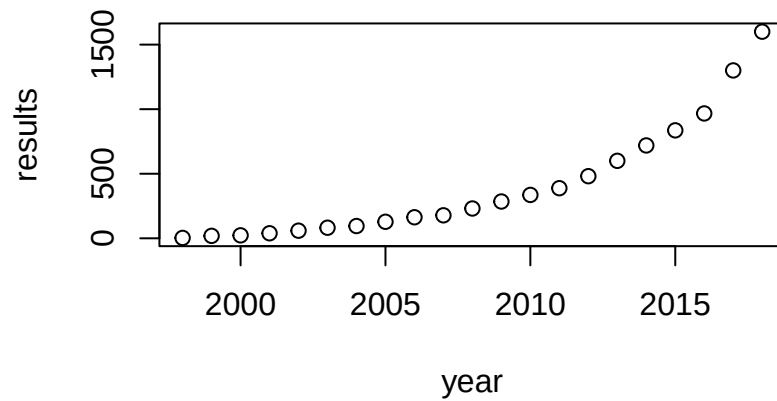
```
summary(social_robot)
```

```
##      year      results
##  Min.   :1998   Min.    :   3.0
## 1st Qu.:2003   1st Qu.:  82.0
## Median :2008   Median : 231.0
## Mean   :2008   Mean    : 406.3
## 3rd Qu.:2013   3rd Qu.: 600.0
## Max.   :2018   Max.    :1600.0
```

Plotting

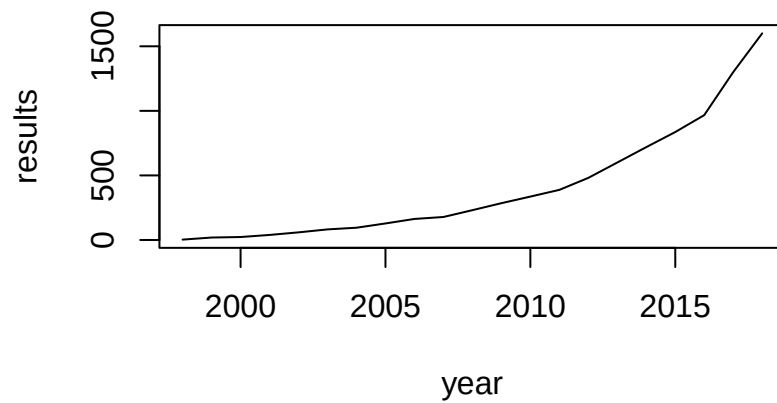
Next, we will plot the year against the count:

```
plot(social_robot)
```



Or better, a plot of type line.

```
plot(social_robot, type='l')
```



`plot` is the most straightforward plotting command. Pull up the documentation of the command and have look at it (`?plot` or `help(plot)`).

We can plot multiple data series by first calling `plot(...)` and each new series with `lines(...)`.

However, for our different data series, this *does not work* because we cannot distinguish the individual data series.

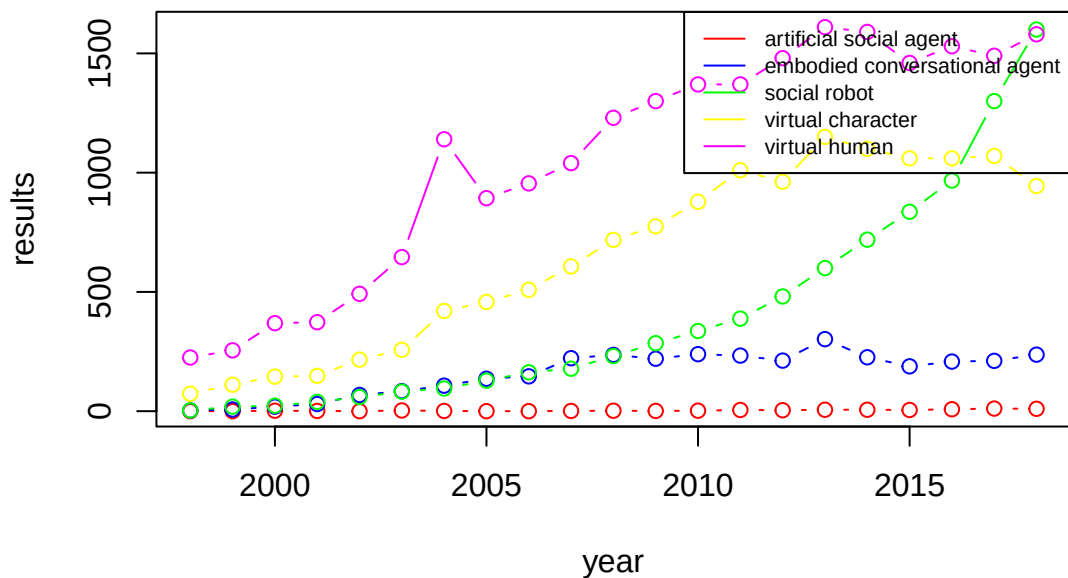
Adding colour and legend

To make this is a useful plot, we need to know which data series is which. Firstly, we can set the individual lines colours with the `col` parameter, e.g. `col="red"`. Do be able to identify the series, we will add a legend using the `legend(...)` command. Do make things easier, we'll create a list with the name of the data series:

```
ll = c('artificial social agent',
      'embodied conversational agent',
      'social robot',
      'virtual character',
      'virtual human')
```

Then we can plot in color and apply the legend:

```
plot(artificial_social_agent,ylim = c(0,1610),col = "red",type = 'b')
lines(embodied_conversational_agent,col = "blue",type = 'b')
lines(social_robot,col = "green",type = 'b')
lines(virtual_character,col = "yellow",type = 'b')
lines(virtual_human,col = "magenta",type = 'b')
legend("topright", legend = ll,
      col = c("red","blue","green","yellow","magenta"),
      lwd = 1, cex = 0.7)
```



Note that because the data series have different ranges of y-values, we need to manually set the y range of the plot.

Exercise 1

As before, plot all the data series that we have imported into a single plot. But, use the `max()` function to determine the maximum value for the plot y-axis and the `rainbow([..])` function to give each line a different colour.

Exercise 2

Explore the different parameters that the `legend` command provides. What is the function of the parameter `cex` and `pch`?

Data wrangling

The solution we have seen so far is not very elegant because we have to treat each data series individually. It would be nicer to find a more efficient way to process the data. As we will see, managing data is an important part of working with R. We will often want to merge, filter, and reshape data.

R has a lot of built-in functionality to manipulate data.

The first thing we want to do now is to create a single data set from the 5 separate data sets. This will make our lives a lot easier down the line.

In principle, we could just paste all the columns of the data files together, because we know that they all have 20 rows that represent the same year. However, this is *not* a *safe* way to handle the data because it makes strong assumptions about the structure of the data. What if we are missing the data for one year in one of the data files? What would be the result of the pasting operation?

The robust (i.e. less error-prone) approach is to use the data itself to do the operation.

The operation of choice is the `merge([..])` command (or one of the varieties provided by different R packages).

To merge three of the five data sets, we need to combine them based on the variable ‘year’ using the `by=` parameter:

```
temp <- merge(virtual_character, virtual_human, by='year')
merge3 <- merge(temp, social_robot, by='year')
```

As you notice, the merging of multiple data sets quickly becomes cumbersome. Moreover, we have lost the name of the data sets in the process:

A more elegant way to merge data

The first thing we want to do, is rename the ‘results’ column for each data set. Unfortunately, this will need to be done by hand:

```
names(artificial_social_agent) <- c("year", "art_social_agent")
names(embodied_conversational_agent) <- c("year", "emb_conv_agent")
names(social_robot) <- c("year", "social_robot")
names(virtual_character) <- c("year", "virtual_character")
names(virtual_human) <- c("year", "virtual_human")
```

Now, we can use the elegant `reduce([])` function to execute the merge in a single step.

```
all_merged <- Reduce(function(x,y) merge(x = x, y = y, by = "year"),
                     list(artificial_social_agent,
                           embodied_conversational_agent,
                           social_robot,
                           virtual_character,
                           virtual_human))
```

year	results.x	results.y	results
1998	73	225	3
1999	111	255	19
2000	145	369	23
2001	148	373	39
2002	216	492	59
2003	257	646	82
2004	420	1140	95
2005	458	893	128
2006	509	955	163
2007	607	1040	178
2008	718	1230	231
2009	775	1300	285
2010	878	1370	336
2011	1010	1370	388
2012	962	1480	481
2013	1150	1610	600
2014	1100	1590	719
2015	1060	1460	836
2016	1060	1530	967
2017	1070	1490	1300
2018	944	1580	1600

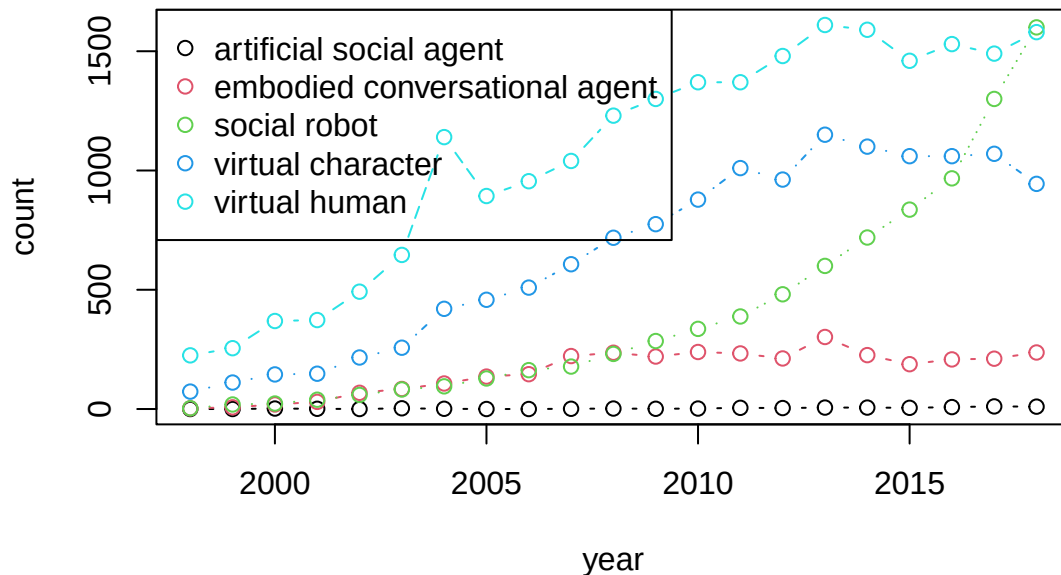
year	art_social_agent	emb_conv_agent	social_robot	virtual_character	virtual_human
1998	0	3	3	73	225
1999	0	7	19	111	255
2000	2	19	23	145	369
2001	1	30	39	148	373
2002	0	68	59	216	492
2003	3	84	82	257	646
2004	1	107	95	420	1140
2005	0	136	128	458	893
2006	0	146	163	509	955
2007	1	222	178	607	1040
2008	2	236	231	718	1230
2009	1	220	285	775	1300
2010	2	239	336	878	1370
2011	5	233	388	1010	1370
2012	4	212	481	962	1480
2013	6	302	600	1150	1610
2014	6	226	719	1100	1590
2015	5	188	836	1060	1460
2016	8	208	967	1060	1530
2017	11	211	1300	1070	1490
2018	10	237	1600	944	1580

Person	Age	Weight
Bob	32	128
Alice	24	86
Steve	64	95

Multiplot

We will now look at a different way in which we can plot the data series with a single command. First off, we use the built-in `matplot()` function. We pass in the column representing the year as the x-value (`all_merged[1]`) and the rest of the columns as the y-value (`all_merged[2:5]`).

```
matplot(all_merged[1], all_merged[2:6], type = c("b"), pch=1, col = 1:5, xlab='year', ylab="count")
legend("topleft", legend = ll, col=1:5, pch=1)
```



Wide vs long (or narrow) format

The layout of the data we have just created is called wide format. Here's the definition from Wikipedia (https://en.wikipedia.org/wiki/Wide_and_narrow_data): 'Wide, or unstacked data is presented with each different data variable in a separate column.'

Compare this to narrow/long/stacked data where 'is presented with one column containing all the values and another column listing the context of the value':

Reshaping the data

Switching from one data layout to another is a fairly common, and sometimes challenging task. One of the strengths of R is its extensibility with packages. Many R packages provided tools for this operation.

We'll now look a different package to convert from wide to long format.

Person	Variable	Value
Bob	Age	32
Bob	Weight	128
Alice	Age	24
Alice	Weight	86
Steve	Age	64
Steve	Weight	95

```
library(reshape2)
all_merged_long = melt(all_merged, id="year")
```

```
library(tidyr)
all_merged_long2 = gather(all_merged, variable, value, art_social_agent:virtual_human)
```

You can now look at the re-shaped data with `View(all_merged_long)` and `View(all_merged_long2)`. The matrix in ‘all_merged_long’ and ‘all_merged_long2’ should be of dimensions (105x3).

The ‘ggplot’ library

One of the most ‘modern’ plotting libraries is ‘ggplot2’. This package makes it easy to plot data in several different ways and gives good control of the details of the graphics. However, in the case of plotting multiple data series, ggplot requires the data to be in a different layout, namely in the long format.

Exercise 3

Using the help page and Google, write the command needed to plot the ‘all_merged_long’ data using the ‘ggplot’ command.